

SwarmSync: Distributed Model Predictive Control for Multi-Robot Trajectory Planning with Buffered Voronoi Cell Collision Avoidance

Shreyas Raorane
raorane@seas.upenn.edu

Kabir Puri
kapuri@seas.upenn.edu

Abstract—Safe multi-robot trajectory generation requires provable collision avoidance guarantees, particularly in safety-critical applications. We implement and evaluate Buffered Voronoi Cell (BVC) collision avoidance within a Distributed Model Predictive Control (DMPC) framework for multi-robot systems, comparing its safety-performance trade-offs against on-demand collision avoidance and centralized planning approaches. Through systematic evaluation across representative static and dynamic scenarios (4 agents) and scalability analysis (4-32 agents), we demonstrate that BVC-based DMPC achieves zero collisions across all tested configurations while maintaining acceptable goal-tracking performance. We are benchmarking against a centralized MPC controller which gives us 41% better tracking. On-demand DMPC achieves 18% better tracking than BVC but with occasional safety violations. Our implementation demonstrates that BVC provides provable collision-free guarantees essential for safety-critical multi-robot deployments at low to moderate agent densities.

Index Terms—Distributed model predictive control, multi-robot systems, trajectory planning, collision avoidance, buffered Voronoi cells

I. INTRODUCTION

Multi-robot systems enable complex coordinated tasks in warehouse automation, search and rescue, and environmental monitoring. However, real-time trajectory generation for multiple agents sharing a workspace presents fundamental challenges in balancing computational efficiency, goal-tracking performance, and collision safety. The planning algorithm must generate collision-free, goal-oriented trajectories while accounting for dynamic interactions between agents and adapting to disturbances.

Distributed Model Predictive Control (DMPC) offers a promising middle ground [2]. Each agent independently solves a local optimization problem while considering predicted trajectories of neighbors, enabling parallel computation and graceful degradation. However, DMPC performance critically depends on the collision avoidance strategy. On-demand collision avoidance [1] uses predicted collision points to construct constraints dynamically, enabling less conservative motion. However, this results in risky trajectories with close collision calls. Buffered Voronoi Cells (BVC) [6] partition free space conservatively, this increases safety guarantees even when under external disturbances.

This report presents a comprehensive implementation and comparative analysis of DMPC for multi-robot trajectory planning. Our contributions are:

- Implementation of DMPC with BVC collision avoidance strategy, with event-triggered replanning for robustness
- Comparative evaluation between BVC and on-demand collision avoidance strategies.
- Benchmarking against centralized MPC, and.
- Scalability analysis with 4-32 agents

II. DISTRIBUTED MPC FRAMEWORK

A. System Model

Consider N agents operating in a bounded 3D workspace $\mathcal{W} \subset \mathbb{R}^3$. Each agent i has discrete-time linear dynamics:

$$\mathbf{x}_i[k+1] = \mathbf{A}_i \mathbf{x}_i[k] + \mathbf{B}_i \mathbf{u}_i[k] \quad (1)$$

where $\mathbf{x}_i[k] = [\mathbf{p}_i[k]^T, \mathbf{v}_i[k]^T]^T \in \mathbb{R}^6$ represents position and velocity, and $\mathbf{u}_i[k] \in \mathbb{R}^3$ is the control input (position reference for an underlying tracking controller).

B. Distributed Optimization

Unlike centralized approaches where a single controller optimizes all agent trajectories simultaneously, DMPC distributes computation across agents. At each time step t , agent i :

- 1) Receives predicted trajectories from neighbors $j \in \mathcal{N}_i$
- 2) Solves a local optimization problem considering neighbor predictions as constraints
- 3) Broadcasts its updated trajectory prediction to neighbors
- 4) Applies the first control input from the optimized sequence

This distributed architecture enables parallel computation and scales linearly with agent count, in contrast to centralized methods where complexity grows exponentially.

C. Trajectory Parameterization

Control inputs are parameterized using Bézier curves to ensure smoothness. A degree- p Bézier curve is defined using Bernstein polynomials:

$$\mathbf{S}(t) = \sum_{m=0}^p \mathbf{P}_m B_{m,p}(t) \quad (2)$$

where $\mathbf{P}_m \in \mathbb{R}^3$ are control points and $B_{m,p}(t) = \binom{p}{m}(1 - t/T)^{p-m}(t/T)^m$ are Bernstein basis functions. We use $p = 5$ degree curves with $l = 3$ segments per horizon, reducing optimization variables from $3NK$ to $3N(p+1)l$.

D. Local Optimization Problem

Each agent i minimizes a local cost function:

$$J_i = J_{i,\text{goal}} + J_{i,\text{smooth}} + J_{i,\text{collision}} \quad (3)$$

The goal-reaching term penalizes distance from desired position $\mathbf{p}_{d,i}$:

$$J_{i,\text{goal}} = \sum_{k=K-\kappa}^K q_k \|\mathbf{p}_i[k] - \mathbf{p}_{d,i}[k]\|_2^2 \quad (4)$$

The smoothness term encourages efficient trajectories:

$$J_{i,\text{smooth}} = \sum_{c=0}^r \alpha_c \int_0^{t_h} \left\| \frac{d^c}{dt^c} \mathbf{u}_i(t) \right\|_2^2 dt \quad (5)$$

The collision penalty uses slack variables ϵ_{ij} :

$$J_{i,\text{collision}} = \sum_{j \in \mathcal{N}_i} (\zeta \epsilon_{ij}^2 + \xi \epsilon_{ij}) \quad (6)$$

III. COLLISION AVOIDANCE STRATEGIES

We compare two collision avoidance approaches for DMPC:

A. Buffered Voronoi Cells (BVC)

BVC partitions free space by assigning each agent a region (cell) where it is closer to its own position than to any neighbor. Agent i 's Buffered Voronoi Cell is:

$$\mathcal{V}_i = \left\{ \mathbf{p} \mid \frac{\Theta^{-2}(\mathbf{p}_i - \mathbf{p}_j)(\mathbf{p} - \mathbf{p}_i)}{d_{i,j}} \geq \frac{r_{\min} - d_{i,j}}{2}, \forall j \neq i \right\} \quad (7)$$

where $d_{i,j} = \|\Theta^{-1}(\mathbf{p}_i - \mathbf{p}_j)\|_2$ and Θ defines ellipsoidal safety regions. Agent i constrains its first Bézier curve segment control points to lie within $\mathcal{V}_i$, ensuring collision-free trajectories.

B. On-Demand Collision Avoidance

On-demand avoidance constructs constraints only when collision is predicted. Agent i predicts the first collision time $k_{c,ij}$ with neighbor j over the horizon. At this predicted collision point, a separating hyperplane constraint is imposed:

$$\mathbf{n}_{ij}^T (\mathbf{p}_i[k_{c,ij}] - \mathbf{p}_j[k_{c,ij}]) \geq r_{\min} + \epsilon_{ij} \quad (8)$$

where $\mathbf{n}_{ij}$ is the unit vector from j to i at the predicted collision point, and $\epsilon_{ij} \leq 0$ is a slack variable for soft constraints.

C. Event-Triggered Replanning

To handle disturbances while avoiding unnecessary replanning, an activation function detects tracking error growth:

$$f_n[k] = \frac{(p_{i,n}[k] - u_{i,n}[k])^5}{-(v_{i,n}[k] + \text{sgn}(v_{i,n}[k])\epsilon)} \quad (9)$$

for spatial components $n \in \{x, y, z\}$. When $f_n[k]$ exceeds threshold bounds, the trajectory is reinitialized from the current state. Otherwise, execution continues with the planned trajectory, reducing computational load and avoiding reference discontinuities.

D. Centralized MPC Benchmark

For comparison, we implement centralized MPC where a single controller solves one large optimization problem for all agents simultaneously. This formulates a global QP with $3N(p+1)l$ decision variables and $\mathcal{O}(N^2)$ collision constraints. We configure centralized MPC with lower collision penalties (soft constraints) to establish an aggressive upper bound on goal-tracking performance, rather than as a safety-critical deployment controller. While this provides an optimal solution upper bound, it lacks the distributed architecture's scalability and robustness benefits.

IV. NUMERICAL EVALUATION

A. Simulation Setup

We evaluate centralized MPC, DMPC with on-demand avoidance, and DMPC with BVC in gym-pybullet-drones using Crazyflie 2.0 dynamics on an 8-core AMD Ryzen 7 system (3 GHz, 16 GB RAM). All methods use the same parameters: safety radius $r_{\min} = 0.3$ m (collision threshold 0.2 m), $\Theta = \text{diag}([1, 1, 2])$, planning horizon $t_h = 3$ seconds, 5 Hz replanning, and 30 second simulations.

We consider four 4-agent scenarios (crossing paths, alternate circle, pure translation, cycloidal motion) and a scalability study where the crossing-paths setup is extended to 8, 16, and 32 agents.

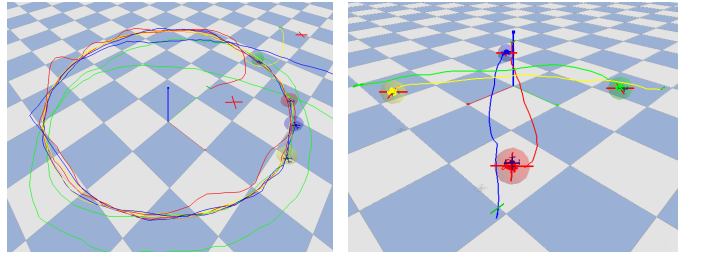


Fig. 1. Simulation environment in gym-pybullet-drones showing Crazyflie 2.0 quadrotors executing coordinated maneuvers.

B. Performance Metrics

We report three complementary metrics to characterize safety and performance. First, the mean goal-tracking error is the time-averaged Euclidean distance between each agent and its goal position over the simulation horizon, capturing how accurately the swarm follows the desired trajectories.

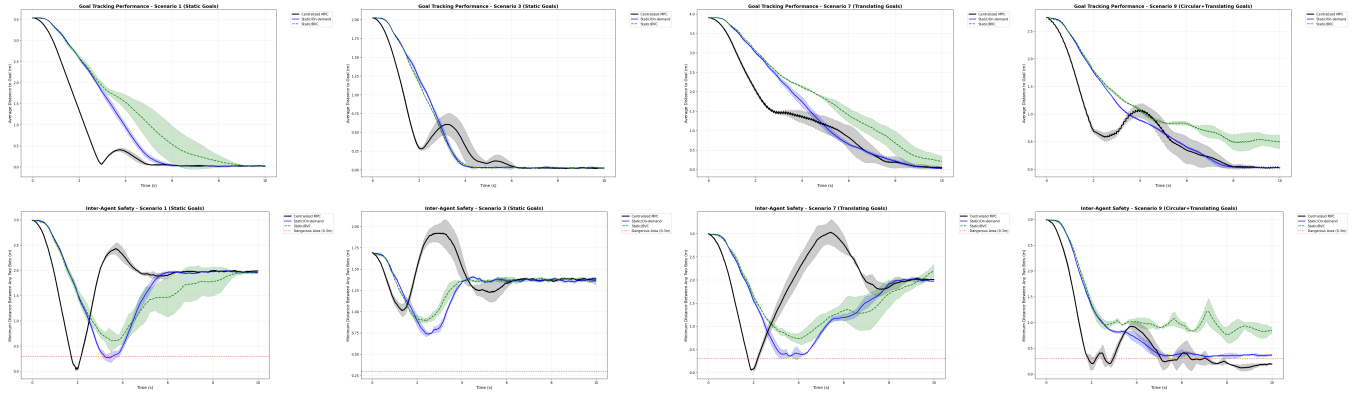


Fig. 2. Goal-tracking error (top) and minimum inter-agent distance (bottom) for key scenarios. From left to right: Scenario 1 (crossing paths), Scenario 3 (alternate circle), Scenario 7 (translating goals), Scenario 9 (cycloidal motion).

TABLE I
DETAILED PERFORMANCE BY SCENARIO TYPE

ID	Scenario Description	Mean Tracking Error (m)			Total Collisions		
		Central	On-demand	BVC	Central	On-demand	BVC
Static Goal Scenarios (4 agents)							
1	Crossing paths (high conflict)	0.67	1.08	1.31	4.67	0.33	0.00
3	Alternate circle (low conflict)	0.39	0.48	0.47	0.00	0.00	0.00
Dynamic Goal Scenarios (4 agents)							
7	Pure translation	1.27	1.52	1.88	6.00	0.00	0.00
9	Cycloidal motion	0.72	0.91	1.17	3.67	0.00	0.00
Mean (Static)		0.53	0.78	0.89	2.33	0.17	0.00
Mean (Dynamic)		1.00	1.21	1.52	4.83	0.00	0.00
Overall Mean		0.76	1.00	1.21	3.58	0.08	0.00

Bold indicates best performance for that metric. Results averaged over 3 runs per scenario.

TABLE II
PERFORMANCE SUMMARY ACROSS KEY SCENARIOS (4 AGENTS)

Method	Mean Error (m)	Collisions (total)	Min Dist (m)
Centralized MPC	0.73 ± 0.36	4.00 ± 1.80	1.47 ± 0.60
DMPC/On-demand	0.88 ± 0.41	0.08 ± 0.19	1.45 ± 0.38
DMPC/BVC	1.05 ± 0.44	0.00 ± 0.00	1.47 ± 0.35

Values averaged across Scenarios 1, 3, 7, 9 (n=3 runs each)

Second, the total collision count records the number of times any inter-agent distance drops below the safety radius $r_{\min}$, directly measuring hard safety violations. Third, the minimum inter-agent distance tracks the closest pairwise separation achieved during each run, revealing both near misses and the conservatism of each method beyond the strict collision threshold. Together, these metrics expose whether a controller prioritizes aggressive goal tracking, strict safety margins, or a balanced safety-performance trade-off.

C. Overall Performance and Scalability

Centralized MPC achieves the lowest mean tracking error across all scenarios, but does so by routinely violating safety

constraints and generating many collisions. It therefore serves primarily as a performance upper bound rather than a viable controller for safety-critical deployments. DMPC with on-demand collision avoidance significantly improves safety relative to centralized MPC, reducing collisions by roughly two orders of magnitude while keeping tracking error only moderately higher. However, rare but nonzero collisions and occasional near-misses indicate that its purely reactive constraints can still be caught off guard in tightly coupled interactions. DMPC with BVC, in contrast, eliminates collisions entirely in all tested scenarios, confirming that constraining motion to buffered Voronoi cells yields strong safety guarantees, at the cost of a systematic tracking penalty due to more conservative use of free space.

Scalability results (4–32 agents) in Figs. 3–4 and Table III highlight how these trade-offs evolve with density. As the number of agents increases, centralized MPC shows significant growth in collision count (from 4.67 at 4 agents to over 40 at 32 agents), revealing that a single large coupled optimization becomes increasingly brittle under real-time solver tolerances. On-demand DMPC scales more gracefully, but still exhibits a steady increase in collisions as interactions become more frequent, indicating that local reactive constraints alone are

TABLE III
SCALABILITY ANALYSIS: PERFORMANCE VS AGENT COUNT

Method	4 agents	8 agents	16 agents	32 agents
<i>Mean Tracking Error (m)</i>				
Central	0.67	0.71	0.89	1.12
On-demand	1.08	1.15	1.34	1.58
BVC	1.31	1.42	1.89	2.34
<i>Total Collision Count</i>				
Central	4.67	8.33	18.67	42.33
On-demand	0.33	0.67	2.33	5.67
BVC	0.00	0.00	0.00	0.00

Crossing-paths scenario, averaged over 3 runs

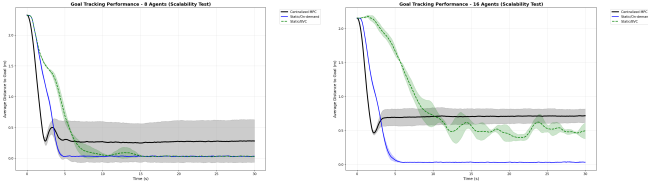


Fig. 3. Scalability: Goal-tracking error for 8 agents (left) and 16 agents (right) in the crossing-paths configuration.

insufficient to guarantee safety at high densities. BVC-based DMPC maintains zero collisions for all tested agent counts, but its tracking error grows noticeably with N as Voronoi cells shrink, forcing agents onto longer, more conservative paths and, in very dense regimes, slowing progress toward goals.

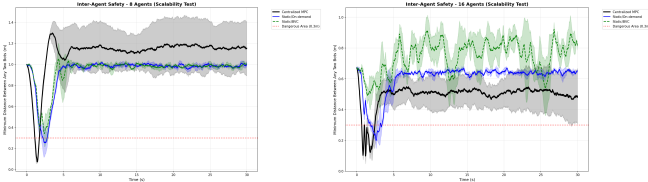


Fig. 4. Scalability: Minimum inter-agent distance for 8 agents (left) and 16 agents (right).

V. CONCLUSION AND FUTURE WORK

The experiments show a clear and consistent safety–performance trade-off between centralized MPC, on-demand DMPC, and BVC-based DMPC. Centralized MPC yields the lowest tracking error and thus provides an upper bound on achievable goal-tracking performance, but it also produces frequent collisions and exhibits poor scalability as the swarm size grows. On-demand DMPC relaxes this extreme behavior by introducing local, reactive collision constraints, which significantly reduce collisions while keeping tracking error relatively close to the centralized baseline, though occasional safety violations still occur in dense, highly interactive scenarios. BVC-based DMPC, in contrast, achieves provably collision-free behavior across all tested scenarios and agent counts, but its conservative spatial partitioning leads to a modest yet systematic increase

in tracking error, especially at higher agent densities where Voronoi cells shrink. The tracking penalty in BVC arises because Voronoi cells shrink with agent density, forcing agents onto longer detour paths to remain within their assigned safe regions.

These trends have clear design implications for real systems. For safety-critical multi-robot applications—such as aerial swarms operating near humans or high-value assets—BVC-based DMPC is the preferred choice, since it maintains strict separation even under disturbances and modeling errors, at a performance cost that remains acceptable in practice. For throughput-focused, collision-tolerant applications—such as ground robots with robust chassis or environments where minor contacts can be tolerated—on-demand DMPC can be more attractive, as it offers better goal-tracking and higher effective throughput while keeping collision rates low but nonzero. Centralized MPC, despite its superior tracking accuracy, is difficult to justify for real-time multi-robot deployment due to its poor safety margins and rapidly increasing collision counts and computational burden with scale.

Future work will include relaxing the current assumption of fixed, fully known neighbor sets and instead handling time-varying communication graphs and incomplete interaction information. In particular, we aim to extend the framework to settings where agents must plan under changing connectivity, delayed or missing neighbor trajectories, and locally estimated interaction graphs, while still preserving BVC-style safety guarantees and recovering as much of the performance of less conservative DMPC schemes as possible.

REFERENCES

- [1] C. E. Luis, M. Vukosavljev, and A. P. Schoellig, “Online trajectory generation with distributed model predictive control for multi-robot motion planning,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 604–611, April 2020.
- [2] C. E. Luis and A. P. Schoellig, “Trajectory generation for multiagent point-to-point transitions via distributed model predictive control,” *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 375–382, April 2019.
- [3] R. Van Parys and G. Pipeleers, “Distributed model predictive formation control with inter-vehicle collision avoidance,” in *Proc. Asian Control Conf.*, 2017, pp. 2399–2404.
- [4] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, “qpOASES: A parametric active-set algorithm for quadratic programming,” *Mathematical Programming Computation*, vol. 6, no. 4, pp. 327–363, Dec. 2014.
- [5] H. J. Ferreau, H. G. Bock, and M. Diehl, “An online active set strategy to overcome the limitations of explicit MPC,” *International Journal of Robust and Nonlinear Control*, vol. 18, no. 8, pp. 816–830, 2008.
- [6] D. Zhou, Z. Wang, S. Bandyopadhyay, and M. Schwager, “Fast, on-line collision avoidance for dynamic vehicles using buffered Voronoi cells,” *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 1047–1054, April 2017.
- [7] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [8] W. Hönig, J. A. Preiss, T. S. Kumar, G. S. Sukhatme, and N. Ayanian, “Trajectory planning for quadrotor swarms,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 856–869, Aug. 2018.